

然而，这样可能会出错，因为它在对等线程的赋值语句和主线程的 `accept` 语句间引入了竞争(race)。如果赋值语句在下一个 `accept` 之前完成，那么对等线程中的局部变量 `connfd` 就得到正确的描述符值。然而，如果赋值语句是在 `accept` 之后才完成的，那么对等线程中的局部变量 `connfd` 就得到下一次连接的描述符值。那么不幸的结果就是，现在两个线程在同一个描述符上执行输入和输出。为了避免这种潜在的致命竞争，我们必须将 `accept` 返回的每个已连接描述符分配到它自己的动态分配的内存块，如第 20~21 行所示。我们会在 12.7.4 节中回过头来讨论竞争的问题。

另一个问题是在线程例程中避免内存泄漏。既然不显式地收回线程，就必须分离每个线程，使得在它终止时它的内存资源能够被收回(第 31 行)。更进一步，我们必须小心释放主线程分配的内存块(第 32 行)。

 **练习题 12.5** 在图 12-5 中基于进程的服务器中，我们在两个位置小心地关闭了已连接描述符：父进程和子进程。然而，在图 12-14 中基于线程的服务器中，我们只有一个位置关闭了已连接描述符：对等线程。为什么？

12.4 多线程程序中的共享变量

从程序员的角度来看，线程很有吸引力的一个方面是多个线程很容易共享相同的程序变量。然而，这种共享也是很棘手的。为了编写正确的多线程程序，我们必须对所谓的共享以及它是如何工作的有很清楚的了解。

为了理解 C 程序中的一个变量是否是共享的，有一些基本的问题要解答：1) 线程的基础内存模型是什么？2) 根据这个模型，变量实例是如何映射到内存的？3) 最后，有多少线程引用这些实例？一个变量是共享的，当且仅当多个线程引用这个变量的某个实例。

为了让我们对共享的讨论具体化，我们将使用图 12-15 中的程序作为运行示例。尽管有些人为了的痕迹，但是它仍然值得研究，因为它说明了关于共享的许多细微之处。示例程序由一个创建了两个对等线程的主线程组成。主线程传递一个唯一的 ID 给每个对等线程，每个对等线程利用这个 ID 输出一条个性化的信息，以及调用该线程例程的总次数。

12.4.1 线程内存模型

一组并发线程运行在一个进程的上下文中。每个线程都有它自己独立的线程上下文，包括线程 ID、栈、栈指针、程序计数器、条件码和通用目的寄存器值。每个线程和其他线程一起共享进程上下文的剩余部分。这包括整个用户虚拟地址空间，它是由只读文本(代码)、读/写数据、堆以及所有的共享库代码和数据区域组成的。线程也共享相同的打开文件的集合。

从实际操作的角度来说，让一个线程去读或写另一个线程的寄存器值是不可能的。另一方面，任何线程都可以访问共享虚拟内存的任意位置。如果某个线程修改了一个内存位置，那么其他每个线程最终都能在它读这个位置时发现这个变化。因此，寄存器是从不共享的，而虚拟内存总是共享的。

各自独立的线程栈的内存模型不是那么整齐清楚的。这些栈被保存在虚拟地址空间的栈区域中，并且通常是被相应的线程独立地访问的。我们说通常而不是总是，是因为不同的线程栈是不对其他线程设防的。所以，如果一个线程以某种方式得到一个指向其他线程栈的指针，那么它就可以读写这个栈的任何部分。示例程序在第 26 行展示了这一点，其中对等线程直接通过全局变量 `ptr` 间接引用主线程的栈的内容。

code/conc/sharing.c

```

1  #include "csapp.h"
2  #define N 2
3  void *thread(void *vargp);
4
5  char **ptr; /* Global variable */
6
7  int main()
8  {
9      int i;
10     pthread_t tid;
11     char *msgs[N] = {
12         "Hello from foo",
13         "Hello from bar"
14     };
15
16     ptr = msgs;
17     for (i = 0; i < N; i++)
18         Pthread_create(&tid, NULL, thread, (void *)i);
19     Pthread_exit(NULL);
20 }
21
22 void *thread(void *vargp)
23 {
24     int myid = (int)vargp;
25     static int cnt = 0;
26     printf("[%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27     return NULL;
28 }

```

code/conc/sharing.c

图 12-15 说明共享不同方面的示例程序

12.4.2 将变量映射到内存

多线程的 C 程序中变量根据它们的存储类型被映射到虚拟内存：

- 全局变量。全局变量是定义在函数之外的变量。在运行时，虚拟内存的读/写区域只包含每个全局变量的一个实例，任何线程都可以引用。例如，第 5 行声明的全局变量 `ptr` 在虚拟内存的读/写区域中有一个运行时实例。当一个变量只有一个实例时，我们只用变量名（在这里就是 `ptr`）来表示这个实例。
- 本地自动变量。本地自动变量就是定义在函数内部但是没有 `static` 属性的变量。在运行时，每个线程的栈都包含它自己的所有本地自动变量的实例。即使多个线程执行同一个线程例程时也是如此。例如，有一个本地变量 `tid` 的实例，它保存在主线程的栈中。我们用 `tid.m` 来表示这个实例。再来看一个例子，本地变量 `myid` 有两个实例，一个在对等线程 0 的栈内，另一个在对等线程 1 的栈内。我们将这两个实例分别表示为 `myid.p0` 和 `myid.p1`。
- 本地静态变量。本地静态变量是定义在函数内部并有 `static` 属性的变量。和全局变量一样，虚拟内存的读/写区域只包含在程序中声明的每个本地静态变量的一个实例。例如，即使示例程序中的每个对等线程都在第 25 行声明了 `cnt`，在运行时，虚拟内存的读/写区域中也只有一个 `cnt` 的实例。每个对等线程都读和写这个实例。

12.4.3 共享变量

我们说一个变量 *v* 是共享的，当且仅当它的一个实例被一个以上的线程引用。例如，示例程序中的变量 *cnt* 就是共享的，因为它只有一个运行时实例，并且这个实例被两个对等线程引用。在另一方面，*myid* 不是共享的，因为它的两个实例中每一个都只被一个线程引用。然而，认识到像 *msgs* 这样的本地自动变量也能被共享是很重要的。

 练习题 12.6

A. 利用 12.4 节中的分析，为图 12-15 中的示例程序在下表的每个条目中填写“是”或者“否”。在第一列中，符号 *v.t* 表示变量 *v* 的一个实例，它驻留在线程 *t* 的本地栈中，其中 *t* 要么是 *m*(主线程)，要么是 *p0*(对等线程 0)或者 *p1*(对等线程 1)。

变量实例	主线程引用的?	对等线程0引用的?	对等线程1引用的?
<i>ptr</i>			
<i>cnt</i>			
<i>i.m</i>			
<i>msgs.m</i>			
<i>myid.p0</i>			
<i>myid.p1</i>			

B. 根据 A 部分的分析，变量 *ptr*、*cnt*、*i*、*msgs* 和 *myid* 哪些是共享的？

12.5 用信号量同步线程

共享变量是十分方便，但是它们也引入了同步错误(synchronization error)的可能性。考虑图 12-16 中的程序 *badcnt.c*，它创建了两个线程，每个线程都对共享计数变量 *cnt* 加 1。

code/conc/badcnt.c

```
1  /* WARNING: This code is buggy! */
2  #include "csapp.h"
3
4  void *thread(void *vargp); /* Thread routine prototype */
5
6  /* Global shared variable */
7  volatile long cnt = 0; /* Counter */
8
9  int main(int argc, char **argv)
10 {
11     long niters;
12     pthread_t tid1, tid2;
13
14     /* Check input argument */
15     if (argc != 2) {
16         printf("usage: %s <niters>\n", argv[0]);
17         exit(0);
18     }
19     niters = atoi(argv[1]);
20
21     /* Create threads and wait for them to finish */
22     Pthread_create(&tid1, NULL, thread, &niters);
```

图 12-16 *badcnt.c*：一个同步不正确的计数器程序